Assignment 1 Report: Frequent Mining Experiments
Q2 (Frequent Subgraphs): gSpan vs. FSG vs. Gaston

COL761/COL7361/AIL7026 Data Mining

February 2, 2026

1 Introduction

Frequent subgraph mining (FSM) generalizes frequent itemset mining from sets to graphs. Given a database of labeled graphs, FSM finds subgraphs that occur in at least a given fraction of graphs (minimum support). Compared to itemsets, the search space is significantly more complex because frequency checking requires subgraph isomorphism (an NP-complete problem).

In this part, we empirically compare three classic FSM tools on the Yeast dataset: **gSpan**, **FSG (PAFI)**, and **Gaston**. We run each method at minimum supports 5%, 10%, 25%, 50%, and 95% and analyze the runtime trends and the algorithmic reasons behind them.

2 Dataset and Preprocessing

2.1 Yeast Dataset

The input dataset is provided in the assignment-specific Yeast format (one graph per protein/structure entry). Using the conversion script `A1/q2/convert_dataset.py`, we parse and convert it into the formats expected by each tool.

From the conversion logs:

- **#graphs:** 64,110
- **#unique node labels:** 11

- **#unique edge labels:** 3

The FSG tool prints additional dataset statistics (from the 10% run header): average of 36 vertices and 37 edges per graph, with maxima of 129 vertices and 135 edges. This indicates that the dataset graphs are moderately sized and sparse.

2.2 Format Conversion and Support Parameterization

The three tools use slightly different conventions for input format and support thresholds:

- **gSpan:** run with `-s` as a fraction in $[0, 1]$ (e.g., 0.10). The script runs `gSpan -f <file> -s <frac> -o`.
- **FSG:** run with `-s` as an integer percentage (e.g., 10 for 10%).
- **Gaston:** expects *absolute* support count. With 64,110 graphs, the corresponding counts are:

  $$\text{supp}(5\%) = 3205,\ \text{supp}(10\%) = 6411,\ \text{supp}(25\%) = 16027,\ \text{supp}(50\%) = 32055,\ \text{supp}(95\%) = 60$$

The experiment pipeline `A1/q2/q2.sh` calls `A1/q2/run_experiments.py`, which first converts the dataset once and then measures wall-clock time around each algorithm invocation. Thus the reported runtimes include the algorithm's own parsing/output overhead for its specific format, but do *not* include the one-time conversion cost.

3 Experimental Results

3.1 Runtime Results

Table 1 summarizes the measured runtimes (seconds). For gSpan at 5% support, the run exceeded the 3600s timeout limit set in the wrapper script; the runtime is therefore reported as 3600s and the output should be treated as incomplete.

MinSup (%)	gSpan (s)	FSG (s)	Gaston (s)
5	3600.00 (timeout)	3558.52	104.21
10	1178.81	1185.07	36.92
25	281.89	336.23	12.51
50	101.90	115.53	6.12
95	7.37	19.60	0.81

Table 1: Runtime comparison on Yeast dataset (lower is better).

3.2 Pattern Output Sizes

To contextualize runtime, we also count how many frequent subgraph patterns are output at each threshold. We estimate this by counting pattern header lines in each output file (lines starting with `t #` for gSpan/FSG outputs and `t` for Gaston outputs).

At 5%, gSpan timed out, so its pattern count reflects only the partial output written before termination.

MinSup (%)	gSpan #patterns	FSG #patterns	Gaston #patterns
5	33,164 (partial)	46,404	46,424
10	8,555	8,549	8,549
25	1,066	1,062	1,062
50	212	208	208
95	5	3	3

Table 2: Number of mined frequent subgraphs at each support (from output files).

3.3 Runtime Plot

Figure 1 shows the runtime curves produced by the pipeline (support on x-axis, runtime in seconds on y-axis), as required by the assignment.

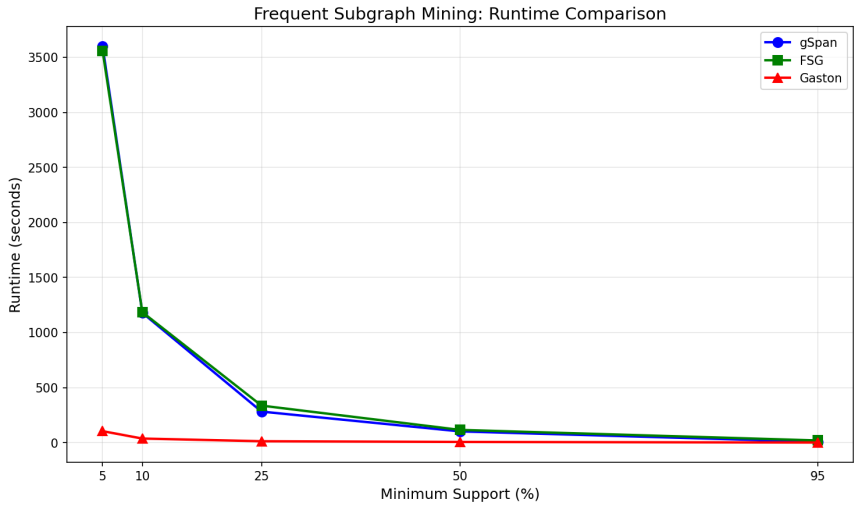

Figure 1: Runtime comparison on Yeast dataset. X-axis: minimum support (%). Y-axis: runtime (seconds).

4 Analysis and Discussion

4.1 High-Level Trend: Runtime Explodes at Low Support

Across all methods, lowering minimum support increases the number of frequent patterns dramatically (Table 2: from 3 patterns at 95% up to ∼46k patterns at 5%). This expands the search space and the number of embeddings that must be tracked, which increases both (i) the enumeration work (how many candidates are explored) and (ii) the verification work (how expensive support counting and canonical-form checks are).

The result is a steep runtime increase for gSpan and FSG in particular: from seconds at 95% to minutes at 50%/25%, and to roughly an hour around 5%.

4.2 Why Gaston is Consistently Faster Here

Gaston is substantially faster than gSpan and FSG at all supports in our runs (e.g., at 10%: 36.92s vs ∼1179s). This gap is most naturally explained by algorithmic specialization:

- **Gaston enumerates paths and trees efficiently:** Gaston decomposes the search into frequent *paths*, then *trees*, and only then general

graphs with cycles. Paths and trees admit much cheaper canonicalization and embedding extension than general cyclic graphs, so a large fraction of the search is handled with relatively lightweight operations.

- **Yeast graphs are sparse:** With an average of 37 edges on 36 vertices, many subgraphs encountered early in the search are tree-like; Gaston benefits disproportionately in such regimes.
- **Reduced overhead per candidate:** Even when the #patterns output is similar across methods (10%/25%/50%), Gaston's per-pattern processing cost is much lower, hence the large constant-factor advantage.

At 5% support, the pattern space is large enough that gSpan hits the 3600s timeout, while Gaston still completes in 104s. This indicates Gaston's pruning and specialized enumeration avoid the worst of the combinatorial blow-up seen in the other two methods.

4.3 gSpan vs. FSG: Similar Curves, Different Search Strategy

In the mid-to-high support regime (10%–95%), gSpan and FSG have comparable runtimes. For example, at 10% both are around 1180s and at 50% both are around 100–115s. However, they arrive there via different algorithmic approaches:

- **FSG (Apriori-like, level-wise):** FSG expands frequent subgraphs by joining smaller frequent subgraphs. At low support, the number of frequent subgraphs per level becomes large, causing expensive candidate generation and repeated subgraph isomorphism/support counting.
- **gSpan (pattern growth with DFS codes):** gSpan avoids explicit candidate generation via joins by growing patterns depth-first and using DFS-code canonical forms to eliminate duplicates. Nevertheless, at low support the number of extensions and projected embeddings becomes huge, and support counting remains costly.

The near-equality in runtime at 10% suggests that, for this dataset and implementation mix, the dominant cost is not only duplicate elimination (where gSpan is designed to help) but the sheer volume of embeddings/support checks required in the frequent regime.

4.4 Support-Specific Observations

- **95% (very high support):** Only a handful of patterns survive (3–5 total). All methods run quickly because the search terminates early; Gaston is fastest (0.81s) due to minimal overhead.
- **50% and 25%:** Pattern counts grow to a few hundred / around one thousand, and runtimes reach minutes for gSpan/FSG but remain seconds for Gaston. This shows that even moderate output sizes can still be expensive when support counting and canonical checks are repeated over many intermediate candidates.
- **10%:** Pattern count is ∼8.5k, and gSpan/FSG runtimes jump to ∼20 minutes. The scaling is clearly superlinear in the number of output patterns, implying a large amount of intermediate search (candidates that are explored but do not necessarily end up in the final output).
- **5% (hardest):** FSG completes just under an hour; gSpan exceeds the 1-hour timeout; Gaston completes in ∼1.7 minutes. This is consistent with the classic "low support = combinatorial explosion" phenomenon in FSM, and highlights Gaston's advantage on sparse graph datasets.

5 Reproducibility

The full pipeline is executed via:

```
cd A1/q2
bash q2.sh <path_gspan_executable> <path_fsg_executable> <path_gaston_executable>
<path_dataset> <path_out>.
```

This script converts the dataset into per-tool formats under `A1/q2/converted_data/`, runs each algorithm at 5%, 10%, 25%, 50%, 95%, saves outputs under the specified output directory, and generates a combined plot `plot.png` with axes labeled in % and seconds.

6 References

- gSpan paper: `https://sites.cs.ucsb.edu/~xyan/papers/gSpan.pdf`
- FSG paper: `https://ieeexplore.ieee.org/document/1316833`
- Gaston paper: `https://liacs.leidenuniv.nl/~nijssensgr/gaston/gaston-april.pdf`